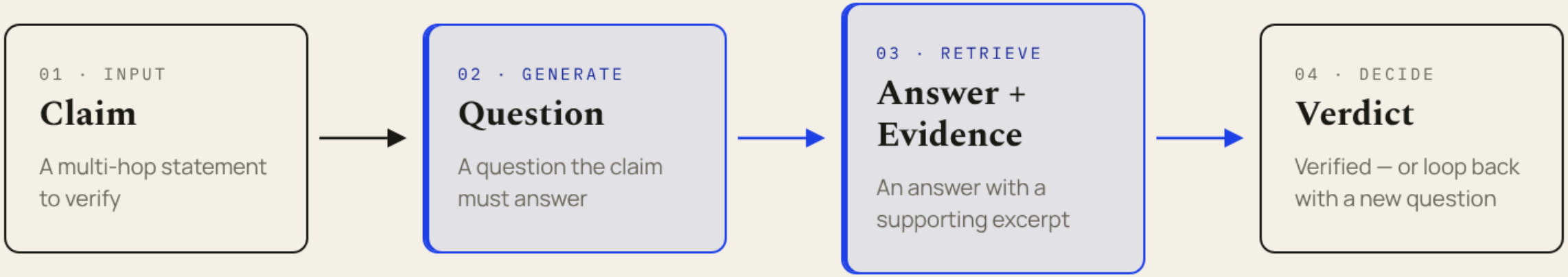
QACHECK: Verifying Claims by Asking Questions

A reading of question-guided multi-hop claim verification — for our prototype decision.



Source: QACHECK (arXiv:2310.07609v1, 2023) · 20-slide methods reading

THE PROBLEM

Multi-hop claims need a chain of facts, not one lookup.

1. A multi-hop claim combines facts that **no single passage states** — verification needs several answers chained together.

2. One-shot entailment or single-question QA **cannot establish the chain**; the intermediate facts stay invisible.

3. Evidence-traceable verification means **every hop** — question, answer, supporting evidence — is inspectable after the verdict.

WORKED EXAMPLE · TWO HOPS

CLAIM

"Sunlight can travel to the deepest part of the Black Sea."

Q1 · QUESTION

What is the greatest depth of the Black Sea?

→

A1 · ANSWER

Black sea has a maximum depth of 2,212 meters.

Q2 · QUESTION

How far can sunlight penetrate water?

→

A2 · ANSWER

Sunlight does not penetrate water below 1,000 meters.

VERDICT

2,212 is greater than 1,000. Therefore, the claim is **FALSE.**

Example: QACHECK paper, Fig. 1.

THE PROBLEM

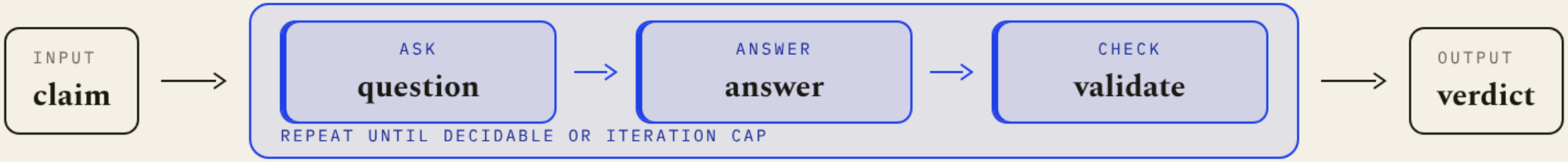
One claim, many hops: verification is a search for a supporting chain.

QACHECK turns that search into a loop of questions.

POSITIONING

QACHECK turns verification into iterative question answering.

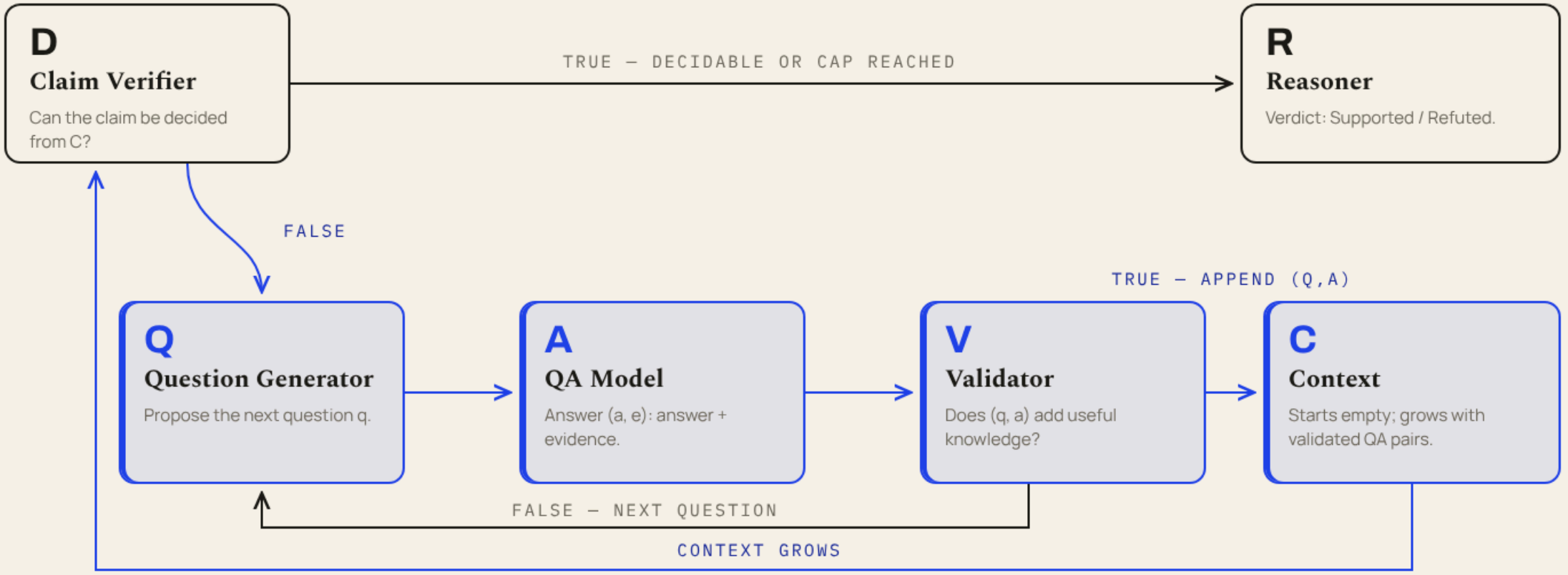
Instead of asking a model to entail the claim in one shot, the system asks questions until the accumulated answers decide the claim.



Direct entailment	one prompt, one verdict, no inspectable chain.
QACHECK	many small QA steps, each answer checked before it joins the context.

MECHANISM

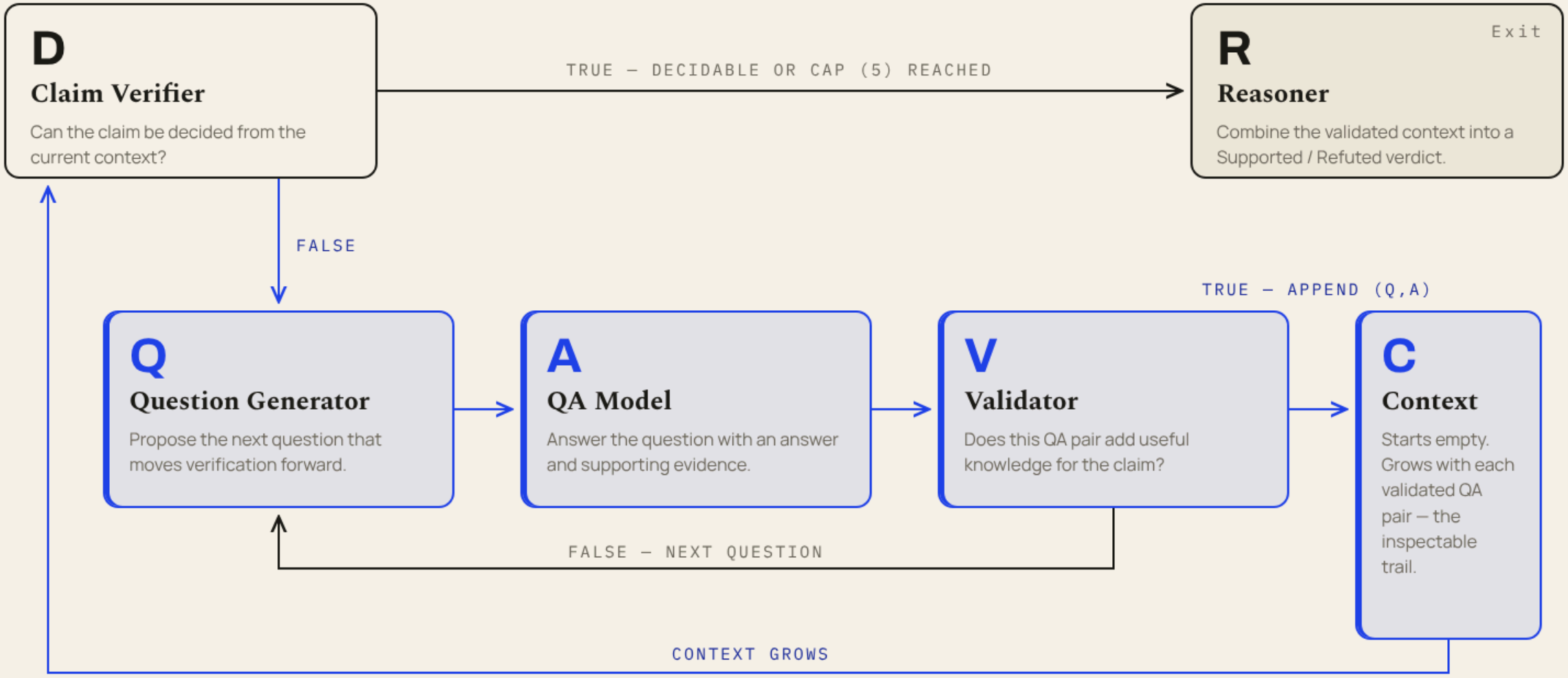
The loop: ask, answer, validate — until the claim is decidable.



Default iteration cap: 5.

ARCHITECTURE

Five components, one job each.



D, Q, V: InstructGPT with 10 in-context demonstrations. A: three swappable implementations (next section).

COMPONENT JOBS · CLOSE-UP

What each component does — and deliberately does not do.

COMPONENT	INPUT		OUTPUT	NOT ITS JOB
D Claim Verifier	(**claim**, context **C**)	→	True / False "decidable now"	NOT ITS JOB Answering questions.
Q Question Generator	(**claim**, **C**)	→	next question q	NOT ITS JOB Judging whether the claim is decided.
A QA Model	**q**	→	(answer a, evidence e)	NOT ITS JOB Deciding the claim.
V QA Validator	(**claim**, {**q**,**a**}, **C**)	→	True / False "useful knowledge"	NOT ITS JOB Generating questions.
R Reasoner	(**claim**, **C**)	→	Supported / Refuted with rationale	NOT ITS JOB Fetching new evidence.

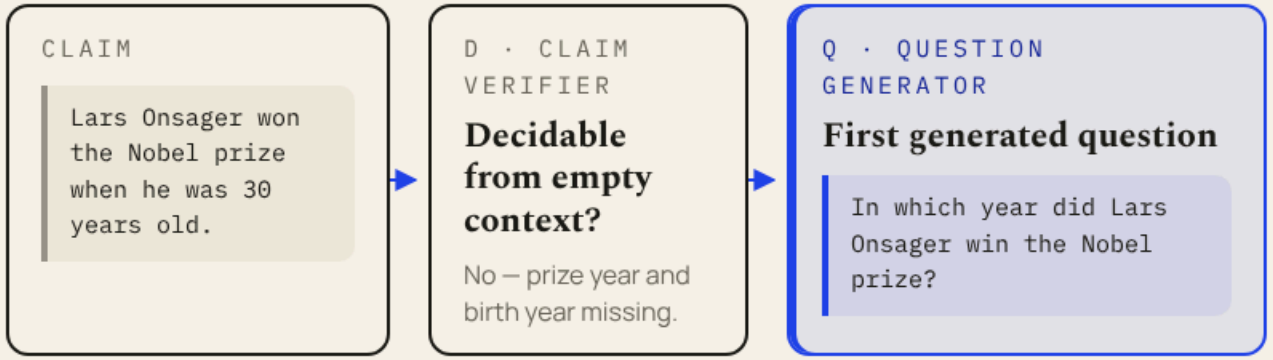
D, Q, V, R default to InstructGPT with 10 in-context demonstrations (Appendix A.1–A.4); R can also be FLAN-T5.

WALKTHROUGH

Follow one claim from question to verdict.

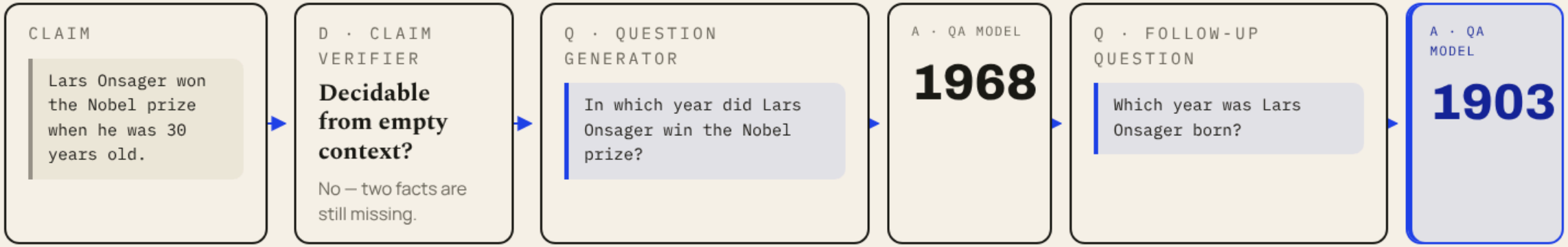
The paper's demo, Fig. 4 — Lars Onsager and the Nobel Prize.

The claim: “Lars Onsager won the Nobel prize when he was 30 years old.”



Demo: QACHECK Fig. 4 · QA model: GPT Reciter-Reader.

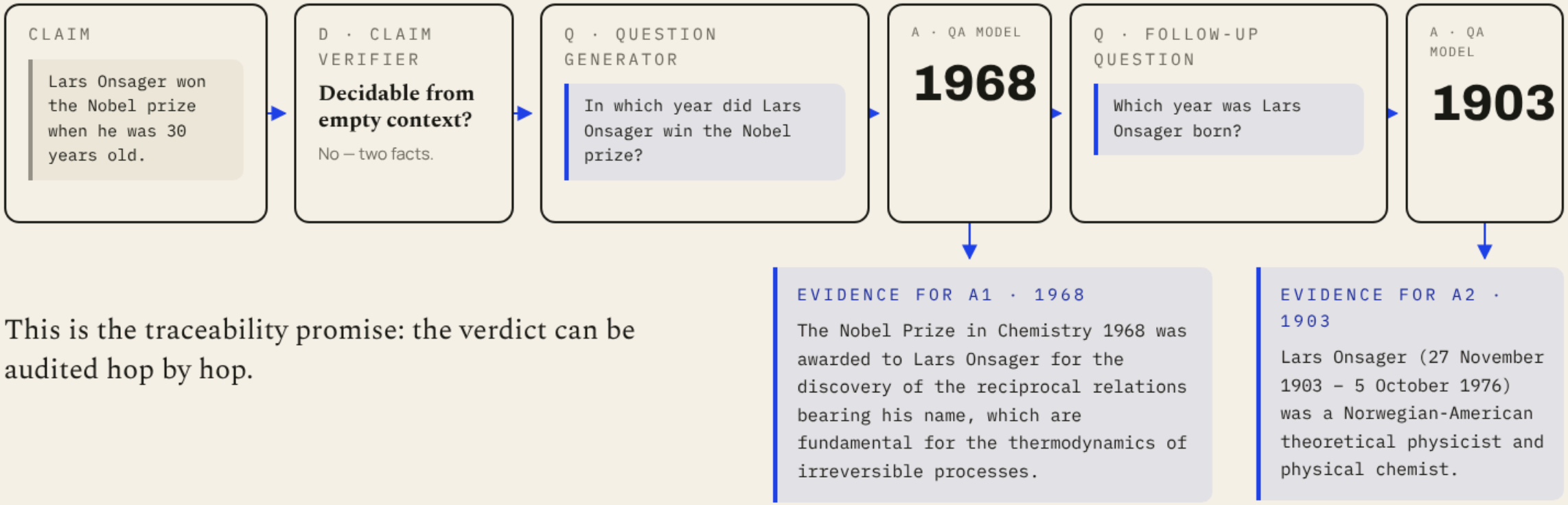
The QA model answers: 1968, then 1903.



Follow-up generation: Q2 is generated only after A1's answer is validated into the context.

Demo: QACHECK Fig. 4 · depths 0 and 1.

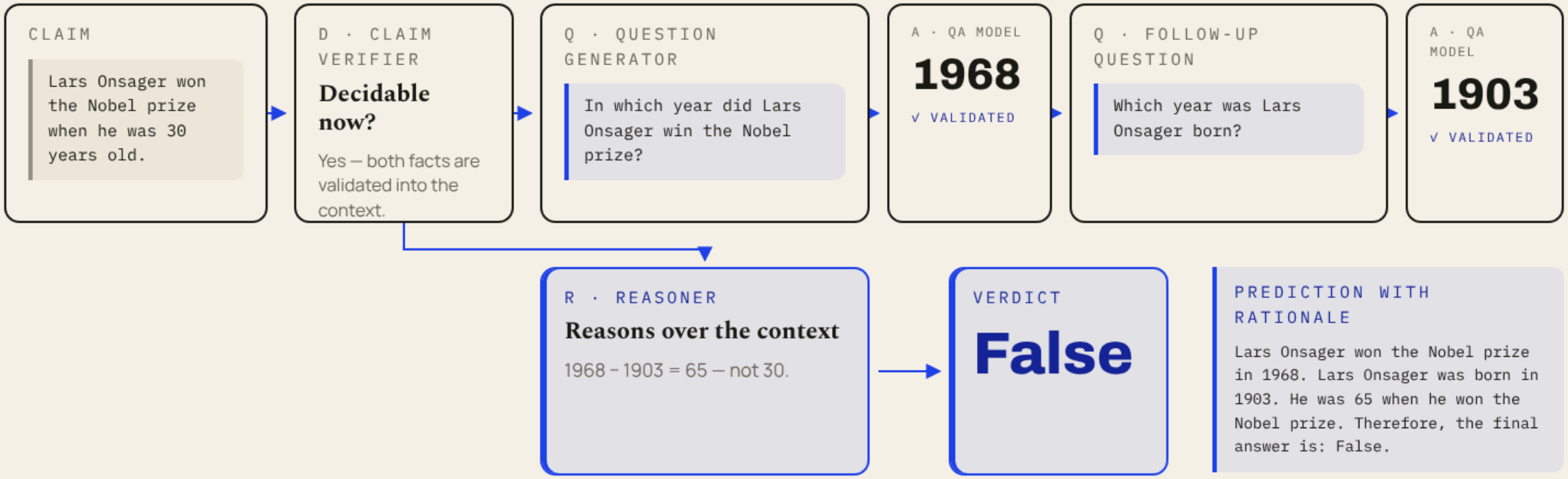
Every answer carries its evidence.



This is the traceability promise: the verdict can be audited hop by hop.

Demo: QACHECK Fig. 4.

The loop closes: 65, not 30 — the claim is False.



Demo: QACHECK Fig. 4 · “Prediction with rationale”.

IMPLEMENTATION

Which question-answering engine should drive the loop?

The loop is fixed; the paper ships three swappable QA implementations.

IMPLEMENTATION

Three QA implementations, one swappable slot.

Retriever–Reader

HOW IT WORKS

BM25 retrieval (Pyserini) over the KILT Wikipedia dump, then a RoBERTa-large reader fine-tuned on SQuAD (PrimeQA).

WHERE EVIDENCE COMES FROM

Retrieved Wikipedia passages.

RETRIEVED EVIDENCE PROVENANCE

FLAN-T5

HOW IT WORKS

Seq2Seq model with parametric knowledge; generates the answer and the evidence directly.

WHERE EVIDENCE COMES FROM

Model-generated — no retrieval.

GENERATED EVIDENCE PROVENANCE

GPT Reciter–Reader

PAPER DEFAULT

HOW IT WORKS

InstructGPT "recites" relevant evidence, then a reader model answers from it.

WHERE EVIDENCE COMES FROM

LLM-recited passages.

RECITED EVIDENCE PROVENANCE

Slot A in the module map; D, Q, V, R unchanged.

QACHECK paper, §QA implementations.

IMPLEMENTATION

What the variants buy: evidence provenance is the real choice.

Retriever–Reader

Evidence is retrieved Wikipedia text — auditable against a fixed corpus.

Quality bounded by retrieval.

RETRIEVED EVIDENCE PROVENANCE

FLAN-T5

Self-contained — no retrieval infrastructure needed.

Evidence is generated, so traceability is weaker.

GENERATED EVIDENCE PROVENANCE

GPT Reciter–Reader

PAPER DEFAULT

LLM-recited evidence with a reader grounding the answer — flexible.

Depends on the reciter's recall.

RECITED EVIDENCE PROVENANCE

READING, NOT A RESULT

The paper's Table 1 reports system-level results, not per-variant breakdowns — variant choice is a design trade-off, not a ranked leaderboard.

EVALUATION

What does the evaluation actually show?

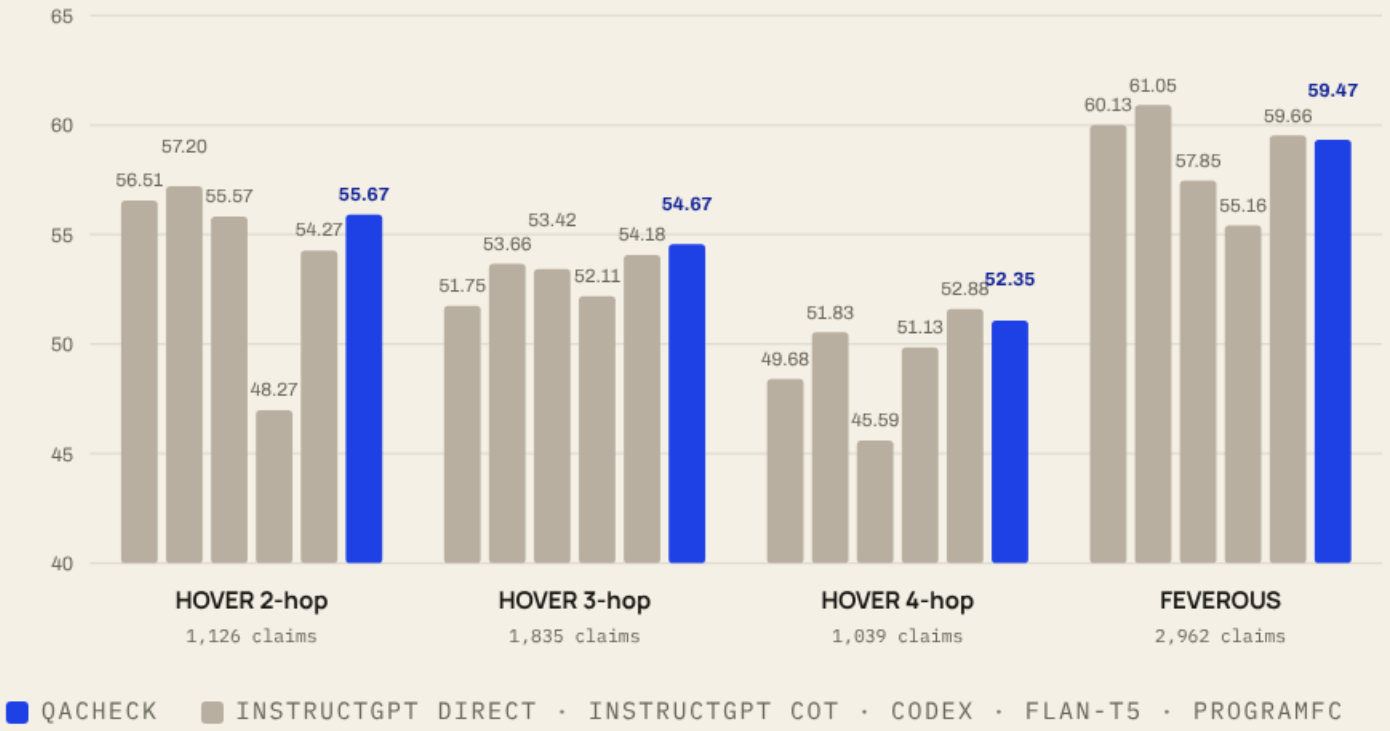
Table 1 · macro-F1 · HOVER 2/3/4-hop and FEVEROUS.

EVALUATION

QACHECK wins one setting outright — and trails the best elsewhere.

Table 1 · Macro-F1 by setting

SIX SYSTEMS × FOUR SETTINGS



QACHECK · INSTRUCTGPT DIRECT · INSTRUCTGPT COT · CODEX · FLAN-T5 · PROGRAMFC

WHAT THE EVIDENCE SUPPORTS

- Beats Direct, Codex and FLAN-T5 in **all four settings**.
- Best on **HOVER 3-hop (54.67)** — its only outright win.
- Paper's own reading: **larger gains over end-to-end models at higher depth**.

HOVER: 1,126 / 1,835 / 1,039 claims · FEVEROUS: 2,962 claims.

ProgramFC, FLAN-T5 and Codex baselines are reported results from Pan et al. (2023), not re-run by the authors.

EVALUATION

What the evidence does not support.

01 No universal superiority

CoT leads on HOVER 2-hop (**57.20** vs **55.67**) and FEVEROUS (**61.05** vs **59.47**); ProgramFC leads FEVEROUS by **0.19** (**59.66** vs **59.47**). QACHECK is on par with both overall.

02 Latency

Reliance on external API LLMs (InstructGPT) prolongs response time; the paper considers LLaMA as a remedy.

03 Label scope

Only True/False claims are handled — Not Enough Information is not.

04 Baseline caveat

ProgramFC / FLAN-T5 / Codex numbers are reported from Pan et al. (2023), not re-run under identical conditions.

For our prototype: QACHECK is a credible architecture, not a proven best system.

PROPOSAL · NOT A RESULT

Does the QA engine matter more than the iteration cap?

HYPOTHESIS

Swapping the QA engine (GPT Reciter–Reader → Retriever–Reader) changes verification quality on HOVER 3-hop **more than** lowering the iteration cap (5 → 3) — because **evidence provenance, not loop length**, drives verdict correctness.

FEASIBLE COMPARISON · ONE DATASET SPLIT, THREE ARMS

SAME CLAIMS (HOVER 3-HOP SPLIT) · SAME D / Q / V / R IMPLEMENTATIONS

A

GPT Reciter–Reader

Iteration cap **5**

PAPER DEFAULT

B

Retriever–Reader

Iteration cap **5**

ENGINE SWAP

C

Retriever–Reader

Iteration cap **3**

CAP LOWERED

OBSERVABLE OUTCOME

macro-F1 per arm, plus **evidence-traceability rate** — the fraction of verdicts whose cited evidence snippet supports the answer.

PRINCIPAL LIMITATION

InstructGPT API cost and latency make Arm A expensive to repeat; results would be a single-lab replication, not the paper's evaluation.

CLOSE

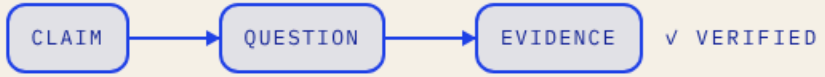
What we take into the prototype.

01 The loop separates asking, answering, validating, and concluding — each hop stays inspectable.

02 The QA engine is the main design lever: retrieved, generated, or recited evidence changes what we can audit.

03 The evidence is mixed: QACHECK is competitive, wins HOVER 3-hop, and is not universally superior.

04 Run the bounded proposal (p. 19) before committing to an engine.



OPEN QUESTIONS

- Which evidence provenance do we need for audit?
- What latency budget do we have?
- Do we need Not-Enough-Information handling?

Verification we can show, hop by hop.